

SpringCloud面试

张伟 (00:01)：我开云录制了。行，放了点儿空间。

codeX (00:06)：好，那我们开始。这个项目首先，使用的微一框架进行二开嘛，整个。整个项目就是充电宝的。怎么说。

codeX (00:43)：直接说代码这个项目是一个若依的二开，二开项目嘛，这个项目里面使用到了小程序，然后还有，Mqtt 以及后台管理系统。使用的话流程就是通过小程序去扫柜机的二维码，然后进行支付宝的使用和归还，可以在。后台管理系统里面看到。充电宝的使用情况。

codeX (01:20)：这样。

张伟 (01:26)：行，我直接开始问问题吧，你说的稍微，

codeX (01:29)：好。

张伟 (01:31)：就整体性的了。

codeX (01:33)：好。

张伟 (01:34)：首先你说一下整个项目的，它的架构和技术选型这块。

codeX (01:40)：首先项目是用的 spring cloud 嘛，然后选型的话是使用的 lsql redis 然后注册中心用的，然后已经使用了消息队列 rabbit mq 嘛。Right.

张伟 (02:02)：没了么。

codeX (02:04)：是不是说他？跟其他竞品的区别，然后再说出为什么选？

张伟 (02:12)：不是你你都用了什么东西，问题特别简单，这个问题就属于是开放性的问题，他的主要的作用其实遇到这种问题对你来说是一个非常好的机会，你想让他问什么，你说什么，你准备的比较好的东西，你就多强调，就这么简单，你就你举个例子，

张伟 (02:38)：你说你你你假如说你 gateway

准备的比较好，或者说你限流准备比较好，你就强调一下？然后重点使用了像 spring cloud gateway，作为我们的整个的，微服务的入口，然后就进行管理，这一块当时做的时候还比较费劲，然后出了一些问题，然后你说，我还通过我们的 sentinel 做了，限流熔断的相关操作和微服务保护的操作，解决了我们线上的一些问题，你看你这么一说就引起他的好奇心，他就肯定追问你到底是怎么玩的。

codeX (03:07)：这样回答！行。

张伟 (03:11)：你首先得先把你用到的东西基本上说个差不多，你不可能只说，然后你到最后你再强调一下，你想要去。

codeX (03:22)：先把所有技术单都说一遍，然后把重要的留到最后来说嘛，然后就延伸下去。

张伟 (03:27)：你可你就不是重要的，而是你准备好的重要不重要，无所谓，

codeX (03:32)：准备好的放到这。

张伟 (03:33)：只要你对，你面试的时候，你你你肯定提前准备了一堆东西。

codeX (03:41)：好，

张伟

(03:41)：你面试的时候提前准备的问题，你那你准备了一堆到最后他一个都不问。你是不是就白准备了？

codeX (03:41)：我知道了。See. 我知道了，面试就这么打。

张伟 (03:56)：对，你面试的很重要的一个点就在于你怎么去引导你的面试官去，把整个面试的方向向着你所期望的方向走，

codeX (04:07)：好。

张伟 (04:08)：你看这里边还有很多你没说的，你例如你像，核心组件里边你像什么分，我们的，数据库也行，包括我们这块涉及到物联网的，它有像 mqtt 协议 emqx 的消息队列。里边还有像我们的 mio 进行对象存储，前端会用到了什么？vue3，用到 uni app，然后数据库里边用的 mysql 用到了 mongodb。用到 redis，你就按分分门别类的给它去捋一捋？整个都用到了啥，然后你再，这里边比较，我当时写的时候比较，有印象的，或者说比较需要去可以注意的？你刚才说的什么什么，你先把整体的给它过一遍，他知道了大体前端用了啥，后端用了啥？中间件用了什么，然后存储用了什么？然后你你里边有没有什么亮点的，你例如你这个项目典型的是一个物联网项目，你这物联网项目的话，你肯定要把物联网的东西要说一说你 mqtt，像 emqx，肯定得说。这些东西，你再最后再去强调你刻意准备的东西。

codeX (05:29)：好好知道了。

张伟 (05:34)：第一个问题，第二个问题。网关在你的项目里面主要承担了什么样的责任？

codeX (05:43)：网关的项目里面主要就是进行一个，你使用一个网关的话，你在前，你在前端调用接口的话，可以调用统一的地址，再根据地址的路径进行分发到各个服务。起到这么一个作用。

张伟 (06:05)：除了还有别的吗？就只是做了一下路由的分发？

codeX (06:13)：还有更核心的就是它起到一个负载均衡的作用。可以对你的接口进行一个分流，不至于说都请求在一个站点上面。

张伟 (06:31)：你这样的就对你的网关的作用理解的不是足够的深刻，整个网关的作用除了你的路由，它里边还会有这样几个点，路由转发肯定是第一块，我们通过我们的断言，用路径去匹配，然后通过我们的 filter，然后一般都是去除一个前缀，去把它负载均衡到不同的微服务上。微服务上，其实是路由的功能，但是，它里边还会有一个，它里面还集成了我们的鉴权。Gateway 里面通常会去鉴权？

codeX (07:03)：对。对。

张伟 (07:05)：然后什么像统一的跨域处理，包括像我们的限流熔断，都会从都会把 center 往这上面去集成，因为它是统一入口嘛。Ok 吧，

codeX (07:16)：好好。

张伟 (07:17)：鉴权，一定要说你 gateway 就是90%~90%以上的 gateway 一定会做鉴权的。

codeX (07:20)：All. 好，

张伟 (07:28)：你这样能引出下一个问题，

codeX (07:28)：那我去。

张伟 (07:29)：他问你，你鉴权是怎么做的？让你聊聊你的鉴权的这一块系统，这块不要忘，

codeX (07:38)：好。

张伟 (07:38)：那你聊一下你的鉴权这块主要包含了哪些功能？或者说怎么设计的？

codeX (07:45)：鉴权这部分就是使用的，若依原生自带的嘛，你请求的时候看你有没有携带 token，然后。没有的话就会进就会提示你，下一步你在，服务之间调用的时候也会有一个鉴权，看你是不是内部服务之间的调用，它也有限制。

张伟 (08:15)：那能不能说一下咱们的整个的权限系统，就包括认证，包括，鉴权这两两步操作分别是怎么做的，就从设计上来说，从实现上来说它的原理是什么？

codeX (08:30)：首先一个请求进来，我在系统里面会首先对它的 token 值做一个判断，看有没有登录。然后还会对它，像在接口上面会有注解进行判断，看它具不具备权限，不具备权限，当然接口也是调不通的。然后。该怎么细说？

张伟 (09:08)：这块能说的东西非常的多，第一块你可以说你的权限的整个的结构的设计？就如例如底层的数据库，表有哪些跟权限相关。再能讲讲我们的 token

的生成，什么时候生成的？然后里边包含了什么东西，再一个我们请求来了之后怎么把 token 带过来，然后你怎么去解析，怎么去判断当前 token 是否有效，怎么从 token 里边去拿到我们的用户信息，角色信息或者权限信息，动态的进行判断，这一系列的都能讲。

codeX (09:40)：好。那我再说一次。首先我们权限使用的是 ibc，在数据库里面会设计，首先会设计一个菜单表，用户表。还有一个角色表，然后你如然后你在登录之后获取到一个 token，会通过 jwt 生成一个 token，然后接口请求之，接口请求进来之后会对 token 进行解析，然后解析到你的用户 id。然后获取到用户 id 之后，我刚刚漏了一下，你生成 token，你生成 token 的时候会同步加入到 redis 里面嘛，然后。token

它会有一个过期时间，在这个时间内你就可以反复的使用，时间过了，你需要重新获取，然后 token 进来之后会对 token 进行一个解析，获取到它的 user id。根据 user id，你可以获取到人的各种权限，然后在我们的接口上面有注解，会看一下这个人是否具有角色，或者说。某个按钮权限，没有的话，当然是调不通的。可以吗？我觉得。

张伟 (11:13)：大体型，但是不是特别的好，就不够具体。就举个例子，你最开始说的表结构设计这一块，不能光说有哪些表表和表之间的关系也得说。再你漏了一张权限表，

codeX (11:27)：行。

张伟 (11:28)：它得有用户角色权限菜单？至少有这些，他们之间的关系应该都是个多对多的关系。

codeX (11:32)：好。

张伟 (11:35)：都要通过中间表去进行维护。

codeX (11:36)：要说多？

张伟 (11:38)：再一个，我们的，再一个就是我们的 token

的生成，你要当然我们先是通过用户密码登录，或者怎么着去通过一个认证的操作，然后给它去生成 token，这个 token 的话，我们里边会包含什么东西？这里边一般来说我们放个 user id？当然，有时候也把权限的信息也放进去，那就看你具体的情况了？看到你自己放不放，都无所谓，

张伟 (12:10)：再来的时候就通过请求头带过来 token

嘛，然后这地方你要说一下，那我们的请求是到网关的。那网关转发到其他的微服务的时候。token 是否还继续带着？

codeX (12:25)：需要带着，不需要带，需要解析到他 user id 就可以了。

张伟 (12:31)：你会把 user id 在内部继续传递，

codeX (12:35)：对，再放大镜头等一下好。

张伟 (12:36)：就这个事儿你，就这个事，你依然也得说，不然后边的微服务咋知道你是谁的呢？

codeX (12:43)：好。

张伟 (12:45)：再一个我们微服务之间互相调用的时候通过分，

codeX (12:45)：好。

张伟 (12:49)：假如说互相调用的时候需不需要鉴权，需不需要认证？是怎么处理的，这块也要说，还有很多的细节，这些细节当然能体现出你到底是一个，能够完成，能够去做的，只是去简单的了解了一下，尽可能的把这些细节说的更具体一些，这样的话，可信度更高，你的都是一些宽泛的，大的这些层面上的东西说的都对，但是，总感觉假假的就那种感觉不够真实，一定要能够到细节才能够体现出，你确实会就假设让你自己去做这套权限系统，你不用恶意，我也能搞得定。得有这种感觉行。

张伟 (13:42)：你这个项目既然用了微服务架构，你说一下，你整个的项目，它如何去划分的微服务，一共有多少微服务？

codeX (13:54)：像我这个项目首先划分的时候基于，首先是去有一个通用模块吗？通用模块的话，里面就会存一些通用的。比如说配置类，或者。一些公用的。就首，首先是配置类，配置里面，首先是一个通用的服务，通用服务里面会存一些通用的配置类，还有一个。等下我捋一下！

codeX (15:05)：重新说一下，我们这个系统会分为，首先是会有一个通用的服务，会有一个，纯 open fee 的服务，还有一个其他具体的功能服务。功能服务就比如说会把划分为用户服务，然后还有。还有订单服务？还有规则服务就类似，然后通用服里面就会存一些通用的配置类，比如说 rabbitmq，然后还有一些 redis 的配置。大体是这样的。

张伟 (15:59)：你这样说就基本上就属于是被判定为对微服务不太懂的那一类型。

codeX (16:06)：对，

张伟 (16:08)：这个回答是非常不好的。首先我跟你说一下你的误区在哪里，

codeX (16:10)：对。

张伟 (16:13)：踩的最狠的一个坑就是基础模块，它不是微服务。分也不是一个微服务，什么叫微服务，你得先明白什么叫微服务，就咱们一个微服务很最基本的或者最简单的评判标准，他得能自己单独跑起来。

张伟 (16:32)：作为服务宝，跑起来部署到服务器上？对外提供接口提供服务的工，能力，你像基础模块它是不会单独部署的，它是作为依赖，作为炸包被别的模块所依赖。

codeX (16:42)：好。

张伟 (16:46)：他就跟你那个妹妹仓库里的一个炸包是没有任何区别的，是你自己写的而已。

codeX (16:52)：所以说通用和笨那些就不用说？

张伟 (16:52)：它是。对它，你说，但你不能说它是微服务，你可以说它是一个基础模块，它是一个份的模块，你是从设计上把它抽取出来了，但是回头运行的时候，它作为其他的模块的一部分。导进去，依赖过去，你还是加了坐标把它引过去？一起去运行的，它不是独立部署，独立运行的微服务。能明白这意思吧，maven maven 中的模块，它不是说每个模块都是一个微服务的，我一个单体项目也可以写100个模块，你拆模块而已？最后还是打一个炸包，那就不是微服务了。微服务一定是单独部署，

张伟 (17:38)：单独打包，单独运维。他甚至会单独的去运营，都是可以的，跟其他的完全独立开来，他自己内部是自制的。对，举个简单例子，没有其他的任何的。微服务，我这一个微服务我也能打个包，

张伟 (17:59)：自己跑，自己去提供服务的能力，自己内部的功能完全是不会丢失的，他不会，他不会说是。跑也跑不起来？然后也打不了包，也没办法自制肯定是不行的，

codeX (18:16)：这样的话？

张伟 (18:16)：第一这你你先听我说完，

codeX (18:19)：好。

张伟 (18:19)：这是第一件事是，第二个坑是什么？你在说微服务的时候，

codeX (18:22)：好。

张伟 (18:25)：没有正面回答我的问题，你们现在有多少个微服务，大体有哪些？怎么去划分的？

codeX (18:30)：好。

张伟 (18:32)：你你说了，首先不是微服务，你说了第二个就是你真正的业务，微服务你也没怎么说，你就说了一个用户的微服务？其他的，

codeX (18:41)：好。

张伟 (18:41)：还有一些其他业务的说明就给人一种感觉，你不熟悉。你你不知道里面都有哪些服务，你举个例子，咱们这里边你像用户服务设备，服务规则，服务，订单服务，支付服务是不是这都是服务，它每一个都是可以单独跑起来的，

codeX (18:58)：好。

张伟 (19:00)：你右键运行，是不是都能单独运行，单独占一个端口，单独能请求的，这些其实都是你在说的的时候，你不用说，所有的都说出来，你至少得说个三四个四五五个的，让你知道这些模块你能说出来的大概率，从面试官的角度你能说出来嘴大概率就是你做的。你要说不上来，你光知道，我们可能有七八个微服务，你说都有啥，说不上来，那说明你就很难去说服人家说你你做过你自己做的模块你都不知道是啥？这是第一个点，你刚才想说的东西你，你可以说一下。

codeX (19:44)：好，这该怎么介绍，比如说我现在重说一下，我的微服务使用了用户服务，然后柜机服务，设备服务和订单服务，用户服务的话接口请求进来。然后就这样说去说吗？

张伟 (20:07)：公众就不用说了，你这些就行了，但是你你你看，问题，这样的，你看我说咱们的微服务是如何划分的？都有哪些微服务？划分的，这也是问题之一，这地方你你这样说，你说我们的微服务，目前来看，目前来说，主要是按业务去划分的。

codeX (20:28)：好。

张伟

(20:30)：有些公司的微服务不是按业务分的，我举个例子，假如说接口就表现层，它单独是一个微服务，

codeX (20:38)：好。

张伟 (20:39)：它可以，它可以把，我们的业务层单独做成微服。把数据库单独做成微服务，各层之间通过分去远程调用，它也有这样的，但是大多数的时候，还是用业务分的比较多 ok

吧？所以说你说等会你为怎么去划分的？

张伟 (20:57)：你说我们按照业务去划分的，你像我们微服务，我现在记得比较熟的有几个？第一个是我们的用户微服务，主要管我们的用户管理，还有认证这些相关的一些操作？然后还有设备服务，我们像什么柜机，充电宝，站点，什么这些服务，这些信息都靠设备服务去维护，还有规则服务，你看我们的计费规则，主要靠规则服务去进行管理，然后订单服务你说像订单，账单，这些，都是通过订单服务去单独去维护的。还有支付服务？你说我们的，跟钱相关的，进行支付，单独去独立的服务出来，你这样一说他就知道，你这一个模块，里边是一个单独的微服务。下面你还要说，

codeX (21:44)：好。

张伟 (21:45)：我们的微服务它有一个设计的原则，我们这一块业务尽量要和其他的要分开，你说我们设计的原则就是什么？我们的服务是高度内聚的，跟其他的模块可能要松耦合。这样去进行拆分，怎么去拆？就举个例子吧，假如说支付服务，因为我们的支付，涉及到比较多的敏感信息，你像什么证书，密钥，跟钱相关的一般都是要求权限也更高一些，数据库的隔离级别也更高一些，数据最好是能够独立去管理，独立自主的，跟其他服务不要互相影响的。你像这种服务，它有特殊的什么安全策略？我们数据，安全级别要求更高，响应速度也要求更快？后期可能我们优先要保的这样的服务，动态伸缩，最好是能够独立出来，我就支付服务，假如说我并发量高了，我可以多上节点。便于我去扩展，这样的话，我们单独设计成微服务，这样的管理起来更加的方便，我支付服务跟其他的服务互不干涉，其他服务挂了也跟我没关系，我支付依然能对外提供服务？

张伟 (23:00)：我自己的数据自己管安全级别更高，不用去，因为过高的安全级别影响其他普通数据的东西。因为你跟别的掺到一起了？它的安全级别也不是说越高越好，那高了肯定就效率低嘛，那有些普通数据，那就单独扔到别的地方去，那我自己管自己的？你这样一说他就能知道了，

张伟 (23:23)：首先你的微服务，整个架构的设计的原则。划分的原则，划分出来哪些，每个里面大体有什么功能，你为什么这么去划分它的理由，就能从一个架构师的角度把这个事就说明白了。他一听就知道，

codeX (23:39)：好。

张伟 (23:39)：你你很内行，这种感觉，他也能因为什么面试官他很难通过一个名字，你写的简历上的几个字，他知道你的公司的业务是什么的，你这么一说，他也大体知道了你公司的整个的业务划分，都有哪些模块，大体有哪些功能，这就很清晰了。

张伟 (23:59)：后边在交流的时候？信息就基本上就能对齐了，就不用他再猜？你们做过什么吗？你做过吗？做过吗？他可以通过这种方式很轻松的跟你去沟通和交流，问你问题啥的也都方便。它是这么个事？

张伟 (24:17)：行，这是，然后再往下，我们微服务架构里面，假如说，我有一个微服务挂了？原因未知，挂了，我如何设计去保证我的微服务，单独的某些微服务，假如说出了一些异常，或者是挂了宕机了，保证它整个系统不受影响的。

codeX (24:44)：首先这就是会一般常的就是 at 和又 a 模式，你要么，好，某个服务挂就是要么你保证他的服务继续可用性，可以让他继续读，然后但不能让他写了，要么你也可以让他继续写，然后最后再去。对他信息进行对齐。

codeX (25:19)：看你使用的是什么模式？

张伟 (25:26)：我在实际开发的时候，我通过什么样的一些技术或者策略，我能够让我的服务，单独一个服务出了问题，其他的整个系统的其他服务不受影响。尽可能的让我们的服务开始保证可用，这么一个，一个问题怎么解决？能说一下具体的方案吗？

codeX (25:50)：我觉得可以，你在服务之间调用的时候，可以做，因为你可以去获取到它是否调用成功吧，如果失败的话，你就，去切换其他节点。都挂了，我觉得就需要去通知一下人为进行。查看一下排查具体原因。

张伟 (26:20)：你这个它更多的是一种运维的手段，我们的，怎么说，就是我们的，从最开始开发的时候，在代码里面，我们或者说从设计的角度来说，怎么去？去预防解决这个问题。

codeX (27:24)：我在想。

张伟 (27:28)：我以为我这没有声音了，

codeX (27:30)：没有，预防问题，挂了是无法避免的？

张伟 (27:43)：对，我们说的是怎么在他挂了之后，其他的服务尽可能不受影响。这个问题，是一个典型的在架构设计上要考虑的高可用问题，整个这个项目对外提供服务，它可能分成若干个微服务？微服务，我们或多或少的可能有些会挂了？你写的有问题，或者是环境有问题，或者压力不一样，等等等各种各样的情况，那万一他挂了怎么办？这时候你把这个问题又转化一下，转化一下。有这么几个点，

张伟 (28:22)：第一个就是从预防手段上我们服务挂了，有可能因为什么原因？但你说写错了，就没办法了，你代码写错了，谁也预防不了？

codeX (28:38)：对。

张伟 (28:39)：我们很多时候预防的什么？是一些。正常的运行过程中的或者运营过程中的一些问题，例如流量过大，并发量过高导致宕机？

codeX (28:50)：去了。

张伟 (28:53)：或者说，用户通过不恰当的方式进行了请求，就本来该请求什么参数的，结果，他搞了一个暴力测试的方式，自己伪造请求。然后给的参数，也不符合我们的要求，导致，它可能接口出问题了？或者说，服务器硬件出了问题，或者网络出了问题，它真的就是宕机了。

张伟 (29:17)：这种情况下，你需要考虑的第一件事是什么？好解决的情况，例如它真的就宕机了，或者说并发量高？这时候我们可以通过什么？你看咱们专门讲过像 sentinel 它就是做微服务保护的。

codeX (29:32)：对，

张伟 (29:32)：宕机了，宕机了我们可以去降级？甚至如果真的出现了一些问题，我们可以去并发量高，我们可以加熔断。

codeX (29:45)：对。

张伟 (29:45)：就容耐降级的策略，通过我们的降级服务，就返回一些兜底数据，这是一种方案？还有，我们微服务之间尽可能一个挂了，另一个影响要小。我举个例子，假如说我订单服务挂了，我支付成功了，但是订单老生成不了，是这意思吧，那首先你要做的是什？

张伟 (30:10)：我们在生成订单这个过程中可以把它做成异步的，我支付成功之后我可以用mq，把支付成功的消息投递到队列中，订单服务再去消费mq的消息，然后去生成订单？它可以做成异步的，不用说是支付完了之后订单也得生成，完了整个的订单的工作才完成，你可以让他们分开，不用说是实时的同步的去请求。不用，直接用分，直接就同步请求他了，支付完了再去分去调订单接口？这样的容易出现级联失败，你用异步的我就投递到里边去，然后订单的挂了，那消息就没人消费了，就在那放着，我们可以暂时不生成订单，等到订单服务恢复了，他可以继续再去消费，继续再生成订单。从用户的角度来说，他的体验可能是什么？支付完了之后订单一直刷不出来，他找客服去问客服说你可以稍等一下，可能服务出问题，等我的订单服务恢复之后，他订单自然而然就出现了。他也不用说是，退掉，然后重新再下单或者怎么着的，这样的能保证他的体验，尽可能的，不是全部的都要去重做。

张伟 (31:28)：你让我们的微服务一个挂了另外的不受影响的最基本的思想就是解耦，他们之间不要关联的太深，他关联的越多。你挂了影响别的越多，跟他没关系的，那肯定他就不会级联的去失败，你用同步的方式去级联掉，就很容易出现任意一个节点出问题，整个业务就挂了。就都垮了，你都得断，你要用分布你，你这样的话一般都会加上分布式事务，你任意一个节点挂了，你所有的都得回滚，等于白做。如果你用异步的方式去通信，你说用mq？那我订单短暂不可用，那就不可用，那支付成功了，告诉大家不用担心钱是丢不了的，订单可能最近服务器压力比较大？订单还没刷出来，等他一会反应过来了，他就刷回来了，那客服还安慰他过一会服服务器一看，出了问题当机，运维那边把它给恢复了之后。他继续去消费咱们的队列中的消息，那订单就出来了。这是一种方案？

张伟 (32:35)：第三个点你还可以说什么？假如说我们用异步的方式通讯？那你说，我们这个地方你就可以提前准备一下，像mq中消息确认或者说消息不丢，怎么保证消息不丢的确认机制？那一大堆。问题是非常，值得去讲的问题，那你在这就可以直接在你看又是个开放性问题？这把它扯出来，

张伟 (33:00)：你说，既然我们刚才说到了mq的异步通信，有可能还出现消息丢失的情况，那我们可以整个的消息通信机制上，加上我们的消息确认机制，重试机制，还有我们的持续化机制？这一堆你给他从头到尾讲讲，确认机制哪地方需要确认，哪地方需要确认，发到网关上丢了怎么办？发到队列上丢了怎么办？队列到消费的时候丢了又怎么办？这一大堆，最后用死心怎么去做兜底？这一块都给他讲讲，

张伟 (33:33)：我们。你像，有些服务，他可能已经挂了，但是还一直在这赖着，一直没剔除，还有人能请求到他，可能微服务集群有5个节点，四个都是正常的。一个挂了。那一个还是能被访问到就容易出现错误，这时候我们可以什么？例如像nacos里边它会有那种不健康的那种节点？自动剔除，你可以把它开启？踢掉，我本来5个我踢掉不行的，那四个能行就用那四个，回头我们恢复好了，这样的话，先让别人别去访问坏了的地方？也是一种方案，又是开放性问题，开放性的问题，你一定要记得，你能想到什么方面，你别管你是用过的没用过的，对的还是错的，你能想到的你都说，你怕你说多了？问题，因为临时问我，我简单思考一下，可能从以下几个方面去说，不一定是非常的契合，咱们探讨一下？

张伟 (34:38)：以下几个点123456。你一说他，他至少能知道你是思考过的，你说有的可能是我的经验，有的可能是我现在临时想出来的一些方案，也没验证过，不一定，但是我都分享出来，咱们一起探讨，你这么一说，你后边就可以放心大胆说了。这种问题一定要多想多说，千万不要两句话，把这个问题给结束掉了，

张伟 (35:02)：开放性的问题，它一般都属于比较难的问题。但是，对我们来说也是一个机会，它能尽可能的让你扯到你熟悉的领域，

张伟 (35:11)：你刚才说的，那我们用分布式事务也是一种方案，但是你要仔细讲讲，你分布式怎么去设计，分布式的模式什么的，稍微放上几个钩子，让你的面试官能够咬上钩之后，顺着你的话题往下捋。别说的特别的含蓄和什么，然后让人听完之后没有问的欲望了，然后就一定要在你的语言里边多设计点钩子，让他去咬钩。是特别重要的，就面试话术这方面，你即使临时想，你要想我怎么用，你好比我就准备了某某技术，你就想他问了这个问题能不能跟技术能扯上，能扯上的话，我就把这个话题往那边去引。他只要是问了。你提前准备的一定答的比你临时发挥的要好得多。行。

codeX (36:09)：好难。

张伟 (36:11)：不是难，你这个意识得转变一下的问题。就为什么你你像你你我举个不恰当的例子，

codeX (36:15)：我觉得答。

张伟 (36:19)：你好比你让我去面试。我可能有些技术我都不会，我都能让面试官说完之后觉着我会，这种能力，你想，你在面试的时候，他是不是非常牛逼的一项能力？

codeX (36:36)：对。

张伟 (36:38)：你能够通过话术引导着你的面试官，去相信你说的话，同时，规避掉你的弱点，强化你的优点，但是从他的角度来说，他没感觉到你的弱点，他就好像你的弱点，你你没说，但是你会一样，你会的，你也能说上来了，你不会的，他那个话题没聊到，他也觉得你会，这样的他就会整体对你的评价高很多，如果问你的问题，你想会的，你说的。也一般，不会的，那更是啥也不说。那你想整个面试下来，他对你的评价就不会高。他是这么个事，行，咱们接着往下，

张伟 (37:22)：刚才这个问题，你如果里边有一些问题，单独的问题，你没来得及记了，你回头可以把。咱录完之后你申请一下权限，回去之后你转转文字看看就行了。

codeX (37:34)：好回去我恶补一下，背一背话术。

张伟 (37:39)：好，下一个问题，你像这个项目里面它是用到了 rabbitmq 的。这个 mq 在什么业务里面去用到了，主要解决什么问题？

codeX (37:59)：像我在我的业务里面，Mq 的使用场景主要，像我进行，用户扫码之后，扫到二维码之后需要去开启柜机，然后开启柜机之后这里会使用 mq 就会告诉，就会告诉系统它已经开启了。然后去确认它开启了，再进行一些操作，用户归还的时候也用 mq 去进行了监听，监听到了用户归还，再用 mq 去生成订单，然后生成订单之后再去，去通过计费规则，然后。看用户多少钱就花了多少钱嘛，然后再去进行支付。使用场景就是这样的，我的项目。

张伟 (38:55)：主要还是服务之间的一些可能是我们的业务的流转，通过这种方式，然后进行异步的解耦？

codeX (39:04)：对。

张伟

(39:06)：行，这个过程中我们可能就会出现像消息丢失这种情况，消息丢失都是在哪些环节可能丢失，

codeX (39:14)：对。

张伟 (39:17)：怎么去解决这些问题。

codeX (39:22)：像在我项目年就比较简单，在他会有个确认机制。因为你生，因为你送 mq 的时候，用户扫完码之后他会，推送一个队列，然后我会今天的队列，我今那个队列。执行完之后我才会确去确认，队内已经执行完，我就上面就只加这么一个圈机制。

张伟 (40:01)：这地方我还是继续说，你这里面尽量不要说的一句话，尽量不要说什么，不要说在我的项目里面，这块比较简单。咱去面试的时候简单的都想吹着，让他特别牛逼，特别难，你你不要他说，东西比较简单，他不简单，他一点都不简单，

张伟 (40:26)：保证消息不丢失，事是非常高难的问题。它需要多个环节的配合才能去解决问题。我在表述问题，我不是教给你吹牛逼，事实上如此，你看咱们消息，首先我跟你讲，我问的问题一般来说，在面试的时候不会问到我这么细，他不会问你什么，哪些地方可能丢，怎么解决，他问你如何保证我们的消息可靠性，或者说怎么保证 mq 消息不丢失。

张伟 (40:54)：就问这么一句话？你说的时候你要分析，首先得分析问题，我们才能解决问题嘛，那容易出现丢失的地方就这些，第一个生产者把消息投递到队列中，可能要经过我们的网关。我们那个 exchange？这些东西，

codeX (41:14)：对交换机嘛？

张伟 (41:16)：对，在到交换机的时候它就可能丢？第二个就是交换机到队列的时候它也可能丢，因为队列它里边消息，因为知道我们默认情况下它是内存里边的？也可能丢，第三种就是我们的队列到消费者的时候它也可能丢，消费的时候可能发过去了，他以为他消费了，没消费？他宕机了，这么几种情况，我们分别用生产者确认机制，生产者往交换机里边去扔的，它会有一个回调，我们可以通过它进行确认，就通过我们的，Correlation data 还有什么？confirm

callback？那两个，然后去完成我们的，到达我们的，消息正式投递到我们的 broker

中，这就相当于是我们的，交换机中。然后第二个，我们可以在。我们的 exchange 和我们的 Q 上面都声明持久化，把消息也设置成 persistent，持久化的模式，能开持久化的都开了，把消息持久化，持久化的意思它可以存到磁盘上，这样的话，即使我们假如说，服务器。不小心？意外宕机了，它不会因为在内存中存储而导致断电丢失，这第二个怎么消息队列里边怎么不丢就持久化，你可以在 exchange 和 Q 上面都生成叫 durable 为 true？

张伟 (42:49)：把它的持久化给它开了。现在很多大多数默认都是 true，但是你要强调一下，有的不。第三个就是消费的时候？我们可以让消费者去严谨一点的，手动去 ack 就确认。手动确认，这样的话，我们把自动确认给它关掉，自动确认就是你只要投递出去了，我不管你消费成功与否都相当于确认了，他没处理。消息就丢了，我们就手动处理，就业务成功之后再手动去 ack，失败了之后，就让他重新入队？没有手动 ack

那没走到确认的代码，就说明业务没完成，那就让他重新入队，这样的话，尽可能保证消息不丢失，

张伟 (43:34)：好，这三块说完是不是？基本上就说完？但是不要忘了，最后补一句，补一句啥，无论我们如何去设计咱们的，项目的这些架构，不能100%，保证消息一定不丢失。能理解我意思吧，因为啥你你不管你的系统设置的多么的完善，它一定会有意外情况发生的。我举个例子，所有的服务器，所有的什么同一时间都被销毁了，都被摧毁了，那你写代码写的再好也没有用，这个地方它其实是这样的，我们说完之后它是这样，我们可以说什么？

张伟 (44:19)：尽量的让运维人员，多备份，多异地备份，保证它的容灾的能力。很多的你像银行，他们的服务器，甚至集群都会布在不同的半球，一些大型的银行，他们的服务器基金会布在不同的半球，他就防止啥？万一打仗了？那这边都被炸了，整个机房都被炸了。那怎么办？你你10个机器去备份10个同时在一个机房里都可能被炸了，它可能东半球放一堆，西半球放一堆，或者南半球放一堆，北半球放一堆，这就有些大型的企业，甚至它把容灾机制会做到这种程度异地备份，你如果整个地球同时被炸了，那整个地球要同时被炸了，我也认了？那人都没了，你还考虑这么多干，所以它这个就是，可以简单聊一句事，你一聊之后，就把整个的话题的氛围就变得轻松了，

codeX (45:18)：好。

张伟 (45:19)：肯定不能100%，咱们只能是尽可能的保证它的不丢失，这些我们，

codeX (45:25)：好。

张伟 (45:25)：从技术上来说，我们能做的基本上就这些了，

codeX (45:29)：好。

张伟 (45:31)：行，下一个问题，mq 的？

张伟 (45:34)：如果我想要 mq 去发延迟消息。都有哪些方案？

codeX (45:41)：延迟消息就两个方案，一个实心队列和一个延迟插件，然后我项目里面使用的是延迟插件嘛，延迟插件。首先你要安装插件，然后在你那个请求里面加上 X delay，就可以作为一个延迟消息了。

张伟 (46:09)：死信队列，它用用来去做这种延迟消息有没有什么弊端？

codeX (46:17)：死神对面我没有死神对面不知道。

张伟 (46:24)：对，这块我跟你说一下，这块有个它有个不算是 bug 吧，它的特性导致的，这个消息，它是同队列的消息，它是先进先出的方式去过期的。你的消息，必须得是前一个过期了之后才判断后过没过期，你懂我意思吗？

codeX (46:45)：先好。

张伟 (46:47)：就举个例子，第一个，假如说你说10分钟后过期，第二个你是5分钟后过期10分钟的，不过期了，后边这5分钟是不是就先过期了？

codeX (46:57)：对。

张伟 (46:57)：但是他不会判断第二个。

codeX (46:59)：还没有推出去？

张伟 (47:01)：他只会把第一个等到10分钟了之后你看，

codeX (47:01)：好。

张伟 (47:04)：过期了扔过去了，然后第二个你看，你5分钟了，早过期了，早过你也扔过去，但是已经晚了5分钟。所以说必须是过期时间单调递增的。这种业务你才能用死因队列？

codeX (47:21)：好。好。

张伟 (47:23)：它不如插件的那种方式更灵活，

codeX (47:27)：好。

张伟

(47:28)：你可以说一下，一般损益队列你很少能保证过期时间是单调增的，一般它。都要谨慎去选择。

codeX (47:40)：好。知道了。

张伟 (47:47)：问一个业务相关的，我们那个扫码，假如说借充电宝，那为啥我不同步的去调用订单接口去创建订单，而用 mq 去异步去处理。

codeX (48:06)：首先，你创建订单，首先是作为一个解耦，如果服务器挂了的话，你这个订单它就不能及时创建成功。所以说通过 mq 去给他做异步的方式嘛，你下次，然后。就过一会儿他，等下我捋一下，

codeX (48:35)：首先就是两个从两个方面考虑，第一个方面。作为异步的可以有，可以有效的减少这个过程的时间，你都是同步的话，它从你生，你去查数据库再生成订单，它就会时间耗费长一些，如果说你是用 mq 做异步的话，用户体验就会好一些，整个过程会更流畅。第二个方面的话就是你使用。Mq 的话可以，有效防止。服务器挂掉，导致用户无法成功创建订单。然后无法支付？

张伟 (49:22)：你说的这两个点就非常的好，就比较完整了，第一个异步的方式，你就跟前端的异步请求也一样？你可以请求之后不用管了，

codeX (49:32)：对。

张伟 (49:33)：让他自己处理去啥时候来了，啥时候回调，这样的用户的体验不中断，他不用一直在那等着，你像我们充电宝？它的弹出操作，它需要快速响应我支付成功了，你尽量就恨不得你一支付成功，你就给我弹出来。但是，我们 mqtt

它去调硬件指令也需要时间。你是同步的话，你看我得先支付成功，再把订单创建成功，再去完成 mqtt 销地投递，再去发指令，再让硬件响应整个过程。既然是同步的，一定是从头到尾先后顺序的，它是一步步阻塞下去的。这样的话，用户的体验可能就会有延迟，半天他弹不出来，即使咱们网络比较好，

codeX (50:23)：对。

张伟 (50:24)：因为你一个接口一个接口请求你请求的接口越多，它肯定就越慢，就影响用户体验，那我就支付成功了，先谈了再说？这是情况，第二个就是我们解耦可以解决那种，例如订单服务，假如说暂时不可用了？我可以消息先积压等待，扔在队列里排着队去，后续服务恢复之后正常再去生成订单就可以了。行。

张伟 (50:56)：下边再问一下，像物联网这块你像 mqtt 你之前了解过吗？

codeX (51:01)：之前没有。

张伟 (51:03)：你你现在能不能简单说一下 mqtt 是什么？

codeX (51:11)：Mqt，我理解就是服务端跟硬件之间交互的一个协议。

张伟 (51:24)：这个协议它的特征是什么？

codeX (51:24)：Yeah. 各种事情。特征不知道。

张伟 (51:35)：你看 mqtt 它是一种基于发布订阅的机制的，通信模型，

codeX (51:41)：Follow. 对。

张伟 (51:42)：他是对他比较适合像一对多的推送的场景。我一次性要，从一大堆的设备上去，获取信息，或者说我一次性给一大堆的设备去发指令，第二个就是 mqg 这种协议，它是一种轻量级的协议。它的消息的格式非常的简单，轻量级它有什么好处？

张伟 (52:11)：我们的物联网设备往往网络状况都不是很好，一般情况下都是什么弄个手机卡啥的就差不多了，你看有些物联网设备，假如说像什么智能井盖，什么智能摄像头都装在很偏僻的地方，网很烂，有的就弄个什么 4g 卡或者 3g 什么，恨不得就两 G 卡流量卡插上物联网卡就用了，它带宽，网络稳定性都很差，那你轻量级的协议传输的数据少，它就。能够在网络差的情况下，保证我们的通信质量传的数据越多，网卡的时候影响越大嘛，

张伟 (52:51)：然后第三个，它有 qos 的级别。什么——一定能送达一次，还是一定能送达多次？怎么着？我们 qs 等于几，有点印象？

codeX (53:06)：我设在里面。

张伟 (53:08)：值不同的时候，它对于消息投递的要求也是不一样的，能够去根据不同的情况来保证我们的消息投递质量。最后一个就是他这种协议还支持你发送遗嘱消息，什么遗嘱消息？设备在异常断线的时候，它会自动的去发送消息，说它断电离线了，

codeX (53:31)：好好好。

张伟 (53:31)：你看像物联网设备，便于我们去检修。这是我们 mqtt，你不用 mqtt，你要么就 http，要么你就自己搭 socket？所以说你不管用哪一个 http 协议，它就太重了。它对网络质量要求太高，因为你经常得握手？你你如果是用 socket 的话，它非常麻烦，你得保持长连接，你的服务器的性能要求比较高，这块，Mqtt 它在物联网上为什么这么火？

张伟 (54:05)：现在我们的物联网通信基本上都用 mqtt 这些东西。是这一点好，那你的像你的我设备上，就你这硬件，都有哪些是通过 mqtt 跟服务器去交互的，然后交互的数据基本上都包含了什么内容？

codeX (54:30)：像我的话跟服务端交互，在他充电宝，充电宝，充电宝使用充电宝和归还充电宝的时候，这两个阶段嘛。我传递给服务器的话会，因为你放，因为你放置柜机的时候手机，首先会根据地点和柜机编号。我再回答一下吧，我贵，我 mqtt 协议，主要是在。充电宝使用和归还的时候和服务端进行通信，因为在放置柜机并装上充电宝的时候，数据库上数据库里会存入它柜机地址。那些插槽信息充电宝的信息，在他们交互的时候会交互的信息，柜机信息他。插槽信息和。充电宝的信息就这就是这三个，可以锁定是哪个，充电宝？

张伟 (55:51)：你这个消息，传递了这么多的信息，有没有你这些信息最后它是怎么去存储的，是存在你的 mysql 数据库中吗？

codeX (56:02)：对的。

张伟 (56:04)：那你设备大约一共有多少台？每一天大约能产生多少数据？问题在物联网的项目中大概率会被问到，

codeX (56:17)：设备多少的？

张伟 (56:21)：就他问你，你因为物联网设备它往往都是一堆设备，你充电宝一样，它肯定不是，这个时候就会涉及到一个问题，你物联网往往假如说是10分钟15分钟或半小时，上报一次数据，那这时候你每一天产生的数据的量，你就得算。

张伟 (56:43)：一天多少数据？一个月多少数据？一年多少数据？你的数据量，你的数据库承载得了，承载不了，否需要分库分表，或者其他的一些处理？这下面要去问的，你的数据量比较大，你直接就这么入库了，有没有去考虑过分库分表问题就这个东西。

codeX (57:04)：保留过什么？

张伟 (57:06)：你看，假如说你你100台机器每个机器上面每分钟，假如说上报5条数据，那你咱拿个计算器出来？你24小时乘以60分钟，这1440分钟，乘以100台机器这些条乘以5个数据，这一天就是72万。

张伟 (57:36)：一个月，乘以30个十百千万十万百万千万一个月就2000多万数据是这意思？

codeX (57:46)：那我该怎么办？

张伟 (57:49)：像我们 mysql 数据库，像 mysql

数据库，你是2000多万的数据，一般情况下就要考虑分表了，就超过百万级别以上的就几百万的。

codeX (58:01)：分秒是什么？

张伟 (58:03)：你要去分表或者是分表，

codeX (58:05)：分别。

张伟 (58:08)：就例如我每个月建一张新的表去存这些数据，因为什么？因为一旦数据过大 mysql 的索引，它不是树形的索引吗？加数嘛，层级可能就会过多，查询效率就会大幅度降低，所以说我们要去分布分表。数据量大了之后，你去在一个库里去存它有时候也会非常影响性能。

codeX (58:41)：意思你每个月，新建一个表，把旧的表就放那就不动了，就当为老数据了嘛。

张伟 (58:50)：是不是说不动了，你该查的去查，这个地方会有单独的技术，可能没有讲，课程里没有讲，例如像沙田 gdbc？它可以去做分库分表，它可以根据你的策略去进行分库分表，然后再根据你的策略，你来查的时候，它会再根据你的策略命中到你对应的库表，然后自动去进行查询，然后再进行结果你的合并。通过像沙丁 gdbc 它是可以做解决，

张伟 (59:18)：它如果有分库分表的，它会继续问你你是怎么分库分表的，用了什么解决方案，有的是硬件分工分表，有的是，通过技术杀电机

gbc，所以说你如果没涉及到过，你就说这块不是我来负责的，给其他的同事做的。

codeX (59:37)：我懵逼了。

张伟 (59:39)：对，所有的问题，它都是可以往下一气往下顺的。

codeX (59:44)：好痛。

张伟 (59:48)：你妈永远也闻不到头。我跟你说一下上面机 abc 你有兴趣可以自己找一找，你这个事你要提前说，你大约有多少设备，你不可能说你做这个项目，你不知道大约有多少设备？你可以说大约有几百个，或者是100来个等等等。你先编一个数，然后自己去估一估。你多长时间上报一次？然后一共有多少数据量？你说我们没分过分表，

codeX (01:00:15)：对。

张伟 (01:00:16)：你可以这样做，我们不往数据库里存，不往 mysql 里存，我往什么存？我往一个时序数据库里存？你 inflas dd tdin，

codeX (01:00:26)：持续又是什么？

张伟 (01:00:30)：这些都是时序数据库。打到咱们群里，实际数据，它有什么好处？它专门是存，按照时间顺序去，大量重复，你像设备，他说句不好听的，你每秒钟传一个又有啥用？大多数的数据格式都差不多，

张伟 (01:00:52)：数据变化量也不大，而且时间是按照时间顺序依次往后排就一直采集，一直采集，一直采集，大多数时候也不查。大多数时候也不查，用时序数据库，它能，它不像关系型数据库结构那么复杂，存储的数据压缩的能力非常强，它能够承载超大型的数据量的一些数据存储。

codeX (01:01:18)：完了，学不到尽头了。

张伟 (01:01:22)：真的是想说，

codeX (01:01:23)：不会写到简历里？

张伟 (01:01:24)：不想说就拉倒了，你不想写就不写这个简历，到最后你想说什么你就写什么，这个咱们一起去是不是我们我给你写，你就必须准备啥，你你你到最后准备到什么程度，咱们就把简历写到什么程度。

codeX (01:01:46)：让我们继续。

张伟 (01:01:46)：再问一个问题，我们的还有 qtt，你像我们 qs，你刚才说你设置成零，设置成零就是最多一次。默认情况下其实就是零，那啥？消息发出去了我就不管了，你你丢了就丢了，你收到了就收到了，它是这么个模式，你看 qs 等于一，

张伟 (01:02:09)：它至少一次。它能保证你的消息是至少能够送达一次的，它有所谓的确认机制，不要确认机制吧，就它一直一直一直发，一直发，直到你收到为止，它可能会重复，然后 qs 等于二。

codeX (01:02:25)：我有时候我收不到。

张伟 (01:02:28)：你看 qs 等于二，恰好一次，他就有确认机制了，他就得握手了，他会严格杜绝所谓的丢包和重复了，他对性能要求非常大，一般很少设置，因为咱的物联网的数据，你想你好比充电宝，我两个小时不给他传数据无所谓。那个电量我晚更新俩小时又能咋地？影响不大。

张伟 (01:02:51)：一般情况下，你像 qs 等于零是符合要求的，有的时候就 qs 等于一，让它至少能发一次，我接下来这个问题就跟 qs 等于一有关，假如说我设成 qs 等于一，我可能会保证它至少有一次能送达，至少有一次，就说明有可能会有多次。他要保证这个消息能送达，他会重复给你发，你这个消息重复发了，你可能就重复去处理？你重复入库，或者怎么着的，我们的 msg 的消息怎么设计能够保证这个消息不重复被消费？解决这个问题？

codeX (01:03:41)：不就是。不是他那啥。因为你。因为你扫了之后然后 mqtt 会？行，因为因为它重复发其实没有关系，因为我系统里面会做监听，然后你。去收到消息的时候，我会判断是不是同一个用户，同一个机子，然后看他有没有是否已经在使用了，都满足，就说明他已经在用了，就不会重复消费了。

张伟 (01:04:37)：是否已经在使用了？是什么意思？

codeX (01:04:41)：因为就是重复消费，那我觉得你都就是现在说的是重复消费这个问题，然后因为你重复发消息过来，然后我系统里面因为会监听嘛，他是否发了消息过来。监听到的话。我就会。

张伟 (01:05:00)：你怎么监听他，他之前发过消息过来？是没有的。

codeX (01:05:05)：对，监听不了，之前的只能说因为我弹出去之后，我的数据库里面会有记录，根据这个来判断。

张伟 (01:05:14)：人家说你要去回库里面去查，是这意思吧？

codeX (01:05:17)：对。

张伟 (01:05:18)：这样的话，你频繁去查数据库，它对性能的影响是比较大的。你每来一条消息得先到库里去查一遍？

codeX (01:05:29)：Now. 那就放 redis 里面？

张伟 (01:05:32)：这是一个好的方案，这是一种好的方案，我们因为 mqtt 消息它都是有唯一的 id 的唯一标注符的，你可以把这玩意给它存到 redis 里？你你通过 redis 去查，然后是否是首次处理，首次处理，

张伟 (01:05:54)：那你就，给直接处理了就行了。是如果之前处理过了，那就不处理了。当然一般情况下推荐通过 redis 的原则操作，

codeX (01:06:01)：对。

张伟 (01:06:04)：保证高并发的情况下被判了失误。行，就说到这儿。我想再说再问点啥？

张伟 (01:06:28)：你的项目里边也用到了 ai 了？

codeX (01:06:32)：他那个 ai 有点太假了。

张伟 (01:06:37)：他就强行往上加的嘛，但是你你你面试的时候其实也是会要去把你的，

codeX (01:06:39)：对。

张伟 (01:06:45)：你们，你比较熟悉的公司的好好比你们公司的业务，你也得去整合到你的项目中。你也得去聊这一部分，当然你你肯定是说的时候就要说的深一点，这地方就有一个问题，你整个的 ai 做了什么业务集成的流程是什么样的？

codeX (01:07:06)：计算流程。让我现在说吗？

张伟 (01:07:11)：你可以准备，现在不说也行，还有像我们的 ai 里面，例如，它里边会涉及到一些跟数据库交互的操作，怎么保证安全性，防范风险？这些东西？这块我再问你一个业务相关的，就你充电里边订单。他那个费用是怎么算的？

codeX (01:07:42)：费用的话，怎么算的？

codeX (01:07:46)：首先我们，首先我们是使用了单独的一个模块去定义规则，规则就使用的 drool 那个规则引擎。在我的项目里面就是五5分钟内就免费，超过5分钟内就开始计费。

codeX (01:08:08)：对。

张伟 (01:08:10)：没一个订单的费用计算就这么简单吗？这个业务你这样说肯定不行，我一个人去借充电宝，到最后我要付钱了，这个钱怎么算出来的？你不可能就这样笼统的这么一说就拉倒了！

codeX (01:08:34)：看你借出时间和归还时间，然后对比记账规则，然后就算出金额。

张伟 (01:08:45)：你这样说就完全规避了你的所谓的规则引擎，这块要聊的话，主要所谓的规则是怎么来的，怎么去算，怎么加载？这些东西其实就是聊一聊规则引擎这个东西就是说我们在订单？它里边会有会存储我们的，什么开始时间，结束时间，就算我们的整个使用的时间，把这个算出来的，时长作为我们规则的调用规则的一个依据，然后通过规则引擎，去计算规则引擎怎么加载？怎么设置？然后怎么怎么算出来的，你这块得大体说一说，最后去算出来我们订单的钱，然后再根据所谓的什么有什么优惠价，什么的，然后再去算出最终的金额返回给我们的。

张伟 (01:09:39)：前端，就是你你要你要把这些点说到你，你这一句话说完了，感觉好像说了什么感，但是又感觉什么都没说，这个业务。那我没做过，我也可以说，就是算出来的。他怎么算的，你要能把细节说出来。能明白我意思吧？

张伟 (01:10:07)：再往下，就回去之后你再梳理，特别业务，当然你不一定非得梳理，你你你可以把这个事梳理一下，你以后简历上面出现的，假如说你比较擅长的项目，你也要把这些业务梳理的细致一些，人家问你这个业务到底是怎么干的？它的流程是怎么怎么走的，你不能很笼统的一句话就说完了，

张伟 (01:10:30)：因为你怎么算的，就这么算了，根据时长就算就完了，这样就没有细节，感觉你就没做过这个东西。我没做过这项，我也知道它是这么算，到底是怎么算的，你要把规则引擎，为什么要引入规则引擎？规则引擎的规则大体是怎么设置的？投多少分钟多少钱，中间多少分钟多少钱？有可能会这样和超过多长时间多少钱？有可能是这样，那我没做，我没写过业务，我作为面试官，

张伟 (01:11:01)：我很好奇？我想问问你这个业务到底是什么样，结果你告诉我就按照规则引擎去算。或者按照时长去算，这个答案很显然令我很失望，我也知道是按照去算，到底是怎么算的，你答完之后我不知道，明白我意思吧？

codeX (01:11:18)：这个是要说出具体的规则还是说。

张伟 (01:11:24)：对，你就是能够把从头到尾？时间是怎么算出来的？根据什么作为依据，通过时间怎么去，怎么去从规则引擎里边拿到规则大体有什么样的规则？为什么我要用规则引擎去管理规则，然后再一个就是最最后的结果怎么出的，这一系列的，你就是从头到尾，

张伟 (01:11:47)：你尽可能的细的要把业务能描述清楚，问业务的时候往往就是问流程图。问业务往往就是问流程图，从开始节点到结束节点，中间都经历了，哪些过程都有可能有哪些分支？你要能说清楚结果你你把中间全忽略了，你就直接开始就到结束了。我问这个问题的意义就没有了，

codeX (01:12:09)：要因为我不记得他那个具体规则，我就知道前5分钟免费，因为我没有支付文件，我就一直用的前5分钟免费去测试。

张伟 (01:12:23)：你实际面试你肯你即使不记得你也得现场，你也得编，你硬编也得编，反正他也不知道是对的还是错的，你就这么说，你不可能就这么简单，就按个时长一算，那句话说就特别的笼统。

codeX (01:12:43)：把它丰富一下。

张伟 (01:12:45)：行，再一个。我用户借了充电宝，他一直没还，在系统中业务是怎么处理的？

codeX (01:12:56)：一直没有还？一直没有还？那就一直计费！

张伟 (01:13:06)：然后他就建完之后他人跑了。不来了？你你记费你记一辈子我也不，我也不交，那这时候怎么办？

codeX (01:13:18)：可爱认识你要不还，首先我们借之前会判断他这个信用分是否达标，达标就不会给押金，不达标就要给押金。押金还走着，没交！

张伟 (01:13:42)：人其实就是考察你这个业务的整个的设计，就不是什么技术问题，问你这个业务有没有设计，怎么设计的，你只要能够自圆其说，说出一个逻辑自洽的都行来，首先你就说有信用体系，那信用分足够的，我们可能会免押金，否则的话我们信用分未知或者是比较低的，我们都可以，都是让他去交押金的，尽可能的避免损失，

张伟 (01:14:06)：第二个，我们长时间不回的，也一定会给他计算，计费规则里面可能要有个封顶，你不可能他。一个月不还你扣他几万块钱？充电宝才多少钱，这显然是不合理的，你设置一个多少钱封顶？例如就押金，假如说99你就99封顶是这意思吧，然后第三个，我引入定时任务，然后我去检测这些未归还的订单，假如说超过。48小时，都没还，那之后我判断为异常，我可以去用微信通知，推送一个消息提醒他，或者说，多次通知之后没有反馈的异常我可以转为人工处理，让客服可以去主动联系他，看看有什

么情况？然后再把信用体系，相应的信用分给它扣除，这些业务就是一些业务的纯业务的东西，它也没有什么什么技术上的东西，或者说能实现不能实现，你只要说了。谁知道你做没做，你说的只要合理他就是对的。

codeX (01:15:19)：反正自圆其说就行

张伟 (01:15:23)：对，他面试的时候，他经常会问，他就想看看你到底这有你很多的一很多人写简历都是瞎编，瞎编一些，这个项目就跑去面试了，一问啥业务细节都不知道。那他大概率他就说你要么技术太菜了，要么你这个东西是假的，无论是哪一种，那显然都对你的面试是不利的。行。别的，别的就没啥了，

张伟 (01:16:01)：可能再问问你，例如你，你整个的工作，整个项目的过程中都遇到过哪些难题怎么解决的，或者你项目里边的亮点都有什么，这都是这些题目，都属于开放式的，给你送钩子的。就让你说你提前准备好的东西了，因为他有时候面来面去，他面的都是一些他想问的问题，你想表达的还没，不一定能表达得到，他会给你一个机会，就问你，你你觉得你还有什么想补充的吗？或者说，你的项目里还有哪些亮点，你在开发这个项目的时候遇到过哪些难题，当时怎么解决的？不一定就有那些难题。但是你一定要去把你想去表达的提前准备好的东西，但是你面试中没涉及到的，你在这块统一的都倒给他。再给自己拉一波分，这个意思，但人问你有什么要补充问的吗？你说没有这一下就给毒死了。当然你是说前面聊的都非常好，他问的问题也都比较到位，你答的也比较到位，你就说没什么其他可聊的了，然后也就还凑合，如果前面你就觉得聊的挺一般，没有什么亮点，你这块可以再找补。Ok 吧。

张伟 (01:17:22)：行，这是。这是。这一块，别的，我这也没啥问了，你看你还有啥要补充的吗？

codeX (01:17:34)：倒也没啥。我到时候准备面试的时候我再详细。问下你。

张伟

(01:17:42)：行，没啥的话，那就这样吧，你再自己好去梳理一下，然后我把后边的再发给你，还有一些 java 可能补充的一些资料我都一并发给你。先这样。

codeX (01:17:57)：拜拜了。拜拜。

重点问题：面试官问，在springcloud gateway里，。网关在你的项目里面主要承担了什么样的责任？我该如何回答？**

在我们的项目中，Spring Cloud Gateway 主要作为所有微服务的统一入口，负责请求的路由转发。

首先，我们根据不同的请求路径将请求转发到对应的服务，并结合注册中心实现服务发现和负载均衡。其次，在网关层统一处理了一些通用逻辑，比如登录认证、Token 校验、权限校验、跨域、请求日志和链路追踪等，避免每个微服务重复开发。

另外，针对接口访问量较大的场景，我们还可以在网关层进行限流、熔断和请求超时控制，提高系统的稳定性。总体来说，网关主要承担统一接入、路由转发和公共能力治理的责任，具体业务逻辑仍然由后端微服务负责。

重点问题：那在springcloud gateway里，该如何去校验token?

我们是在网关层通过全局过滤器统一校验 Token。请求进入 Gateway 后，先判断当前路径是否属于登录、注册等白名单接口；如果不是，就从 **Authorization** 请求头中获取 **Bearer Token**。

如果使用 JWT，网关会验证 Token 的签名、过期时间、签发者以及必要的权限信息，而不是只解析 Token 内容。校验失败就直接返回 401，不再转发；校验成功后，可以提取用户 ID、角色等信息，通过请求头传递给后端服务，然后继续执行路由转发。

在实现上可以使用自定义 `GlobalFilter`，也可以使用 Spring Security WebFlux Resource Server。Gateway 的 `GlobalFilter` 可以对匹配到的路由统一执行过滤逻辑；Spring Security 也支持对 Bearer JWT 校验签名、`exp`、`nbf` 和 `iss` 等字段。

重点问题：所以熔断和降级是怎么在 gateway 里进行实现的？

我们给需要保护的路由配置 `CircuitBreaker`

过滤器。正常情况下，请求会转发到下游服务；如果下游服务频繁超时、抛异常，或者返回连续的 5xx 状态码，熔断器会统计失败率。

当失败率达到阈值时，熔断器从关闭状态变为打开状态。此时后续请求不会再调用下游服务，而是直接进入网关本地的降级接口，快速返回一个友好的兜底结果。等待一段时间后，熔断器进入半开状态，放少量请求去探测下游服务；如果恢复正常就关闭熔断器，否则继续打开。~~~Yaml spring: cloud: gateway: routes:

- id: order-service

uri: lb://order-service predicates:

- Path=/order/**

filters:

- name: CircuitBreaker

args: name: orderCircuitBreaker fallbackUri: forward:/fallback/order statusCodes:

- 500
- 502
- 503
- 504

resilience4j: circuitbreaker: instances: orderCircuitBreaker: slidingWindowSize: 20 # 最近 20 次请求作为统计窗口 minimumNumberOfCalls: 10 # 至少请求 10 次后才开始判断 failureRateThreshold: 50 # 失败率达到 50% 时熔断 waitDurationInOpenState: 10s # 熔断 10 秒后进入半开状态 permittedNumberOfCallsInHalfOpenState: 5 ~~~

降级接口放在 Gateway 本身：

```
@RestController
public class FallbackController {

    @RequestMapping("/fallback/order")
    public Mono<Map<String, Object>> orderFallback() {
        return Mono.just(Map.of(
            "code", 503,
            "message", "订单服务暂时不可用，请稍后重试"
        ));
    }
}
```

```
}  
}
```

重点问题：鉴权这块主要包含了哪些功能？或者说怎么设计的？

鉴权主要分为两部分：认证和授权。认证是确认“你是谁”，授权是判断“你能做什么”。

在我们的设计中，用户先通过登录接口完成认证，认证服务校验账号密码后生成Token；客户端之后每次请求都在 **Authorization** 请求头中携带 Bearer Token。

请求到达 Gateway 后，网关会先处理白名单接口，再校验 Token 的签名、有效期和状态。校验失败直接返回 401；校验成功后，从 Token 中提取用户 ID、角色等基础身份信息，转发给下游服务。

权限控制通常采用 RBAC 模型，也就是用户—角色—权限：一个用户可以拥有多个角色，角色关联菜单和接口权限。网关可以做粗粒度的接口拦截，例如没有登录不能访问；具体到某个业务操作或数据范围的细粒度权限，则由业务服务自己校验，避免只依赖网关。

我们的RBAC是使用的七张表

- 7张表（用户表、角色表、权限表、菜单表、用户角色中间表、角色权限中间表、权限菜单中间表）
- 张三登录系统---> 查询张三拥有的角色列表--->再根据角色查询拥有的权限 ->

再根据权限查询拥有的菜单

- 整体都多对多的关系

你这个项目既然用了微服务架构，你说一下，你整个的项目，它如何去划分的微服务，一共有多少微服务？

我们是通过业务进行划分的 我们的微服务它有一个设计的原则，我们这一块业务尽量要和其他的要分开，你说我们设计的原则就是什么？我们的服务是高度内聚的，跟其他的模块可能要松耦合。这样去进行拆分，怎么去拆？就举个例子吧，假如说支付服务，因为我们的支付，涉及到比较多的敏感信息，你像什么证书，密钥，跟钱相关的一般都是要求权限也更高一些，数据库的隔离级别也更高一些，数据最好是能够独立去管理，独立自治的，跟其他服务不要互相影响的。你像这种服务，它有特殊的什么安全策略？我们数据，安全级别要求更高，响应速度也要求更快？后期可能我们优先要保的这样的服务，动态伸缩，最好是能够独立出来，我就支付服务，假如说我并发量高了，我可以多上节点。便于我去扩展，这样的话，我们单独设计成微服务，这样的管理起来更加的方便，我支付服务跟其他的服务互不干涉，其他服务挂了也跟我没关系，我支付依然能对外提供服务？

面试官：简述Java垃圾回收机制？（GC是什么？为什么要GC）

候选人：

嗯，是这样~~

为了让程序员更专注于代码的实现，而不用过多的考虑内存释放的问题，所以，在Java语言中，有了自动的垃圾回收机制，也就是我们熟悉的GC(Garbage Collection)。

有了垃圾回收机制后，程序员只需要关心内存的申请即可，内存的释放由系统自动识别完成。

在进行垃圾回收时，不同的对象引用类型，GC会采用不同的回收时机

面试官：强引用、软引用、弱引用、虚引用的区别？

候选人：

嗯嗯~

强引用最为普通的引用方式，表示一个对象处于有用且必须的状态，如果一个对象具有强引用，则GC并不会回收它。即便堆中内存不足了，宁可出现OOM，也不会对其进行回收

软引用表示一个对象处于有用且非必须状态，如果一个对象处于软引用，在内存空间足够的情况下，GC机制并不会回收它，而在内存空间不足时，则会在OOM异常出现之前对其进行回收。但值得注意的是，因为GC线程优先级较低，软引用并不会立即被回收。

弱引用表示一个对象处于可能有用且非必须的状态。在GC线程扫描内存区域时，一旦发现弱引用，就会回收弱引用相关联的对象。对于弱引用的回收，无关内存区域是否足够，一旦发现则会被回收。同样的，因为GC线程优先级较低，所以弱引用也并不是会被立刻回收。

虚引用表示一个对象处于无用的状态。在任何时候都有可能被垃圾回收。虚引用的使用必须和引用队列Reference Queue联合使用

面试官：对象什么时候可以被垃圾器回收

候选人：

思考一会~~

如果一个或多个对象没有任何的引用指向它了，那么这个对象现在就是垃圾，如果定位了垃圾，则有可能被垃圾回收器回收。

如果要定位什么是垃圾，有两种方式来确定，第一个是引用计数法，第二个是可达性分析算法

通常都使用可达性分析算法来确定是不是垃圾

面试官：JVM 垃圾回收算法有哪些？

候选人：

我记得一共有四种，分别是标记清除算法、复制算法、标记整理算法、分代回收

面试官：你能详细聊一下分代回收吗？

候选人：

关于分代回收是这样的

在java8时，堆被分为了两份：新生代和老年代，它们默认空间占用比例是1:2

对于新生代，内部又被分为了三个区域。Eden区，S0区，S1区默认空间占用比例是8:1:1

具体的工作机制是有些情况：

-) 当创建一个对象的时候，那么这个对象会被分配在新生代的Eden区。当Eden区要满了时候，触发YoungGC。
-) 当进行YoungGC后，此时在Eden区存活的对象被移动到S0区，并且当前对象的年龄会加1，清空Eden区。

-) 当再一次触发YoungGC的时候，会把Eden区中存活下来的对象和S0中的对象，移动到S1区中，这些对象的年龄会加1，清空Eden区和S0区。
-) 当再一次触发YoungGC的时候，会把Eden区中存活下来的对象和S1中的对象，移动到S0区中，这些对象的年龄会加1，清空Eden区和S1区。
-) 对象的年龄达到了某一个限定的值（默认15岁），那么这个对象就会进入到老年代中。

当然也有特殊情况，如果进入Eden区的是一个大对象，在触发YoungGC的时候，会直接存放到老年代

当老年代满了之后，触发FullGC。FullGC同时回收新生代和老年代，当前只会存在一个FullGC的线程进行执行，其他的线程全部会被挂起。我们在程序中要尽量避免FullGC的出现。

面试官：讲一下新生代、老年代、永久代的区别？

候选人：

嗯！是这样的，简单说就是

新生代主要用来存放新生的对象。

老年代主要存放应用中生命周期长的内存对象。

永久代指的是永久保存区域。主要存放Class和Meta（元数据）的信息。在Java8中，永久代已经被移除，取而代之的是一个称之为“元数据区”（元空间）的区域。元空间和永久代类似，不过元空间与永久代之间最大的区别在于：元空间并不在虚拟机中，而是使用本地内存。因此，默认情况下，元空间的大小仅受本地内存的限制。

面试官：说一下 JVM 有哪些垃圾回收器？

候选人：

在jvm中，实现了多种垃圾收集器，包括：串行垃圾收集器、并行垃圾收集器（JDK8默认）、CMS（并发）垃圾收集器、G1垃圾收集器（JDK9默认）

面试官：Minor GC、Major GC、Full GC是什么

候选人：

嗯，其实它们指的是不同代之间的垃圾回收

Minor GC 发生在新生代的垃圾回收，暂停时间短

Major GC 老年代区域的垃圾回收，老年代空间不足时，会先尝试触发Minor GC。Minor GC之后空间还不足，则会触发Major GC，Major GC速度比较慢，暂停时间长

Full GC 新生代 + 老年代完整垃圾回收，暂停时间长，应尽力避免

重点问题：设计模式

你可以围绕“登录方式”这样回答：

我之前负责过登录业务，最开始所有登录方式都写在一个业务类里，例如密码登录、短信登录、微信登录、一键登录、钉钉登录都通过 if-else 判断。随着需求增加，这个类越来越复杂，新增登录方式也需要不断修改原有代码，维护成本比较高。

后来我使用了策略模式和工厂模式进行重构。首先定义统一的登录策略接口，例如 `LoginStrategy`，然后分别实现密码登录、短信登录、微信登录等具体策略。每种登录方式只负责自己的登录逻辑，避免业务逻辑互相耦合。

然后通过工厂根据登录类型选择对应的策略。工厂负责“选择和获取哪种登录策略”，策略负责“具体怎么执行登录”。调用方只需要传入登录类型，不需要知道具体实现。

这样新增一种登录方式时，只需要增加一个策略实现，并注册到工厂中，原有登录流程基本不需要修改，符合开闭原则，也提高了代码的可维护性和可扩展性。

可以再补一句体现你对两种模式的理解：

简单来说，策略模式解决的是“不同登录方式如何实现”的问题，工厂模式解决的是“根据登录类型如何找到对应实现”的问题。在这个场景中，两个模式是配合使用的。

如果面试官问“为什么不只用策略模式”，可以回答：

只使用策略模式也可以，但调用方仍然需要自己判断并创建具体策略。引入工厂后，可以把策略的选择和创建逻辑集中管理，调用方只依赖接口，职责更加清晰。

再严谨一点，你还可以说明：

如果工厂内部使用 `switch` 或 `if-else` 来选择策略，严格来说更接近简单工厂；在 Spring 项目中，我通常会使用 `Map<LoginType, LoginStrategy>` 自动注册策略，避免工厂本身随着登录方式增加而频繁修改。

面试时最核心的三句话是：

策略模式负责封装不同登录算法。工厂模式负责根据类型获取对应策略。
两者结合，解决登录方式经常变化导致业务类不断修改的问题。